Informe laboratorio 5
Grupo 3

Mariana Cediel 202321548
Valeria Martínez Vélez 202410228
Sofia Rodríguez Chaparro 202222588

Este informe documenta la auditoría técnica realizada sobre el servidor web desplegado en el segmento DMZ (Caso Aura University), como parte del Laboratorio 5 del curso de Ciberseguridad. El objetivo fue identificar vulnerabilidades en la aplicación expuesta, analizar su impacto y aplicar medidas de endurecimiento y mitigación preservando la disponibilidad del servicio.

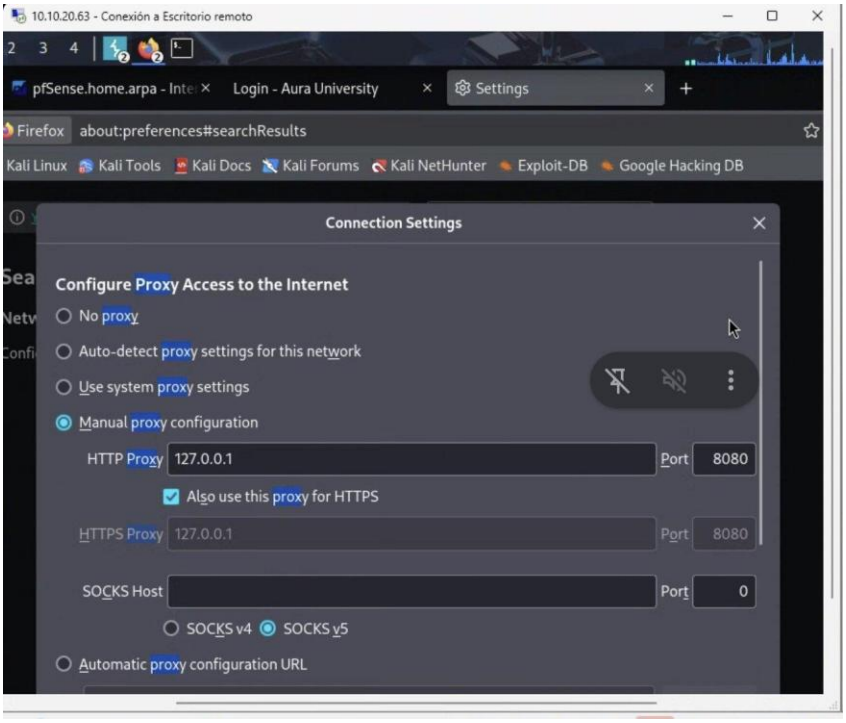


Figura 1.1. Firefox con proxy manual 127.0.0.1:8080 (también para HTTPS) para que Burp Suite capture el tráfico.

1. Reconocimiento de red

La Kali de trabajo está en 192.123.99.0/24 (misma red que la interfaz LAN de pfSense, 192.123.99.1). El descubrimiento sobre ese segmento solo mostró pfSense y la propia Kali, por lo que se revisó la configuración de pfSense para identificar la interfaz OPT1 (192.123.10.1/24), que corresponde al segmento DMZ. Un nuevo barrido sobre ese rango ubicó el host 192.123.10.10 (DMZ.10).

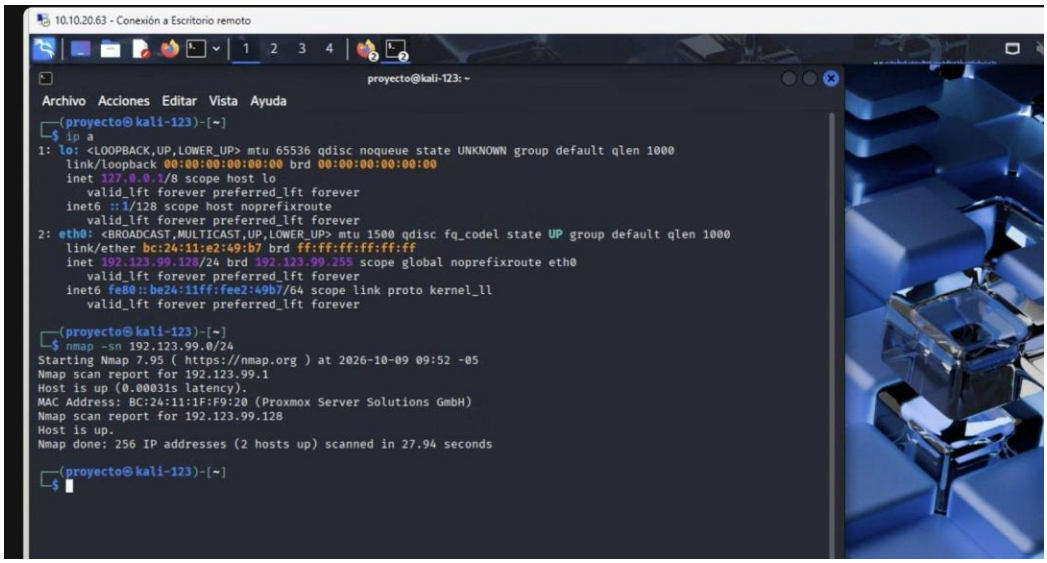


Figura 1.2. Configuración de red de la Kali (192.123.99.128/24) y descubrimiento en 192.123.99.0/24: solo pfSense y Kali.

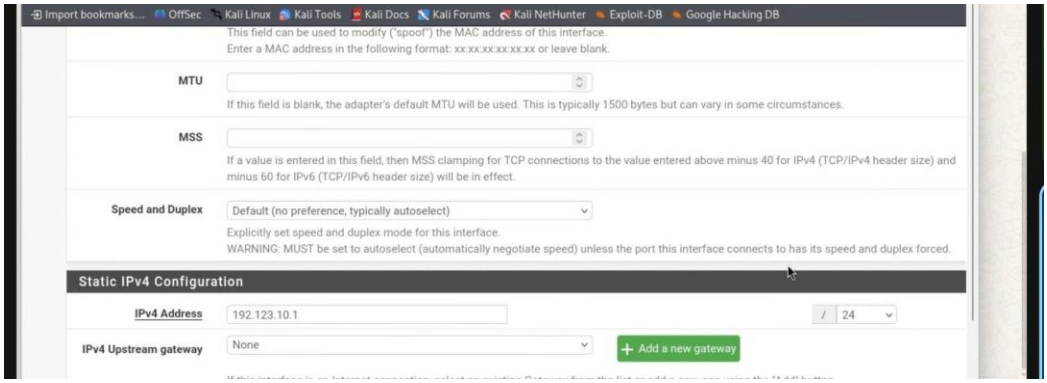


Figura 1.3. Interfaz OPT1 de pfSense: segmento DMZ 192.123.10.1/24.

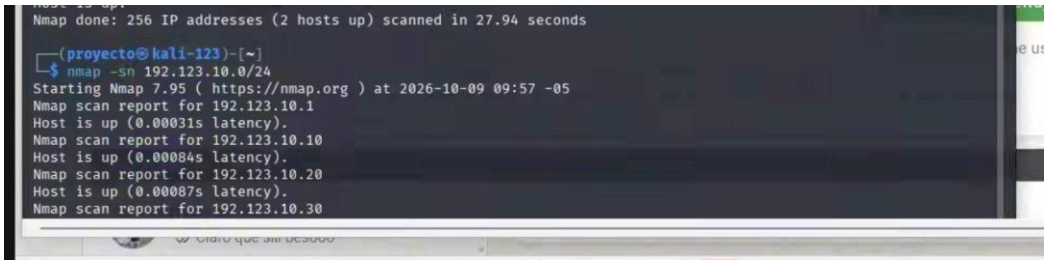


Figura 1.4. Descubrimiento en 192.123.10.0/24, se ubica el host 192.123.10.10 (DMZ.10), objetivo del análisis.

Pruebas adicionales sin resultado

Se probó manualmente una inyección SQL clásica en el login (usuario ' OR '1'='1' --). La aplicación respondió "Credenciales inválidas", por lo que **no se encontró SQLi explotable por esta vía**, se documenta como prueba negativa y no como hallazgo.

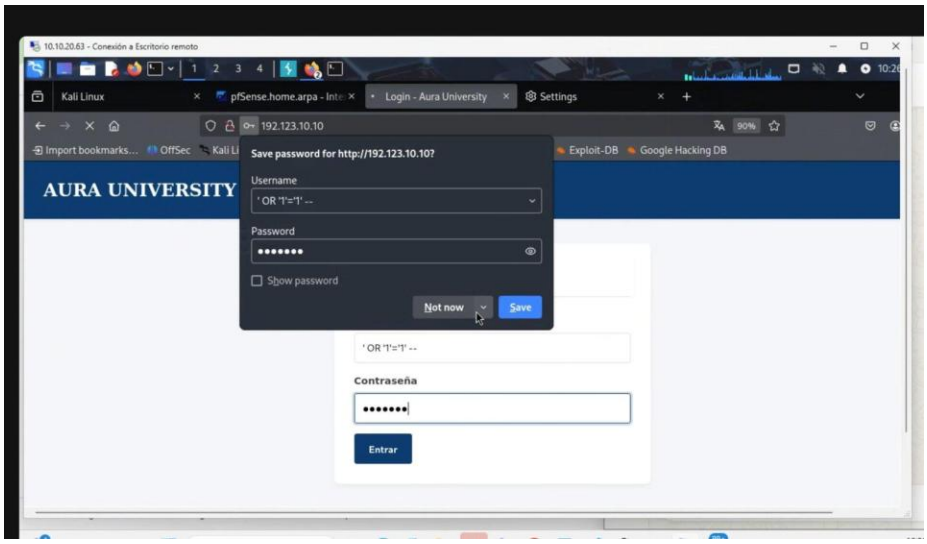


Figura 1.5. Intento de inyección SQL en el formulario de login.

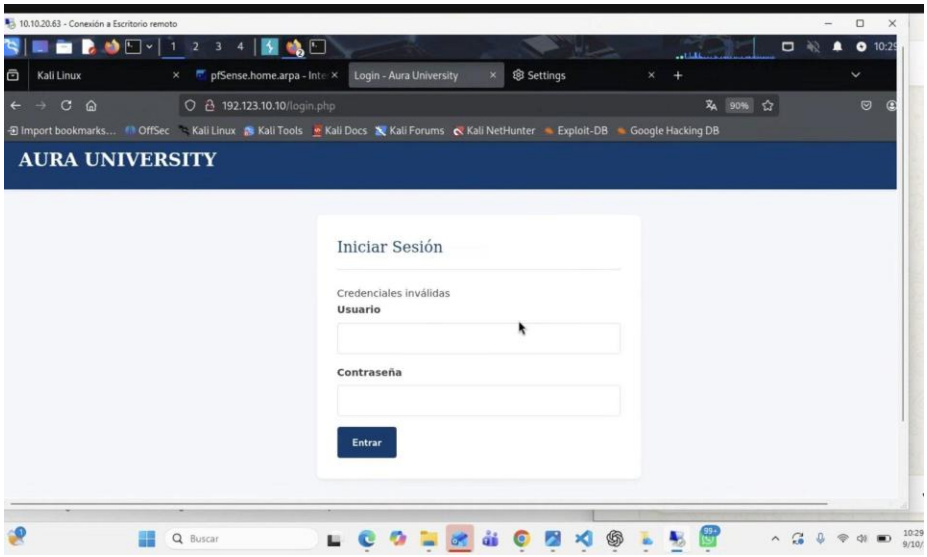


Figura 1.6. Resultado: "Credenciales inválidas" (la inyección no tuvo efecto).

2. Vulnerabilidades identificadas

Vulnerabilidad 1: Transporte no cifrado (ausencia de HTTPS)

Vector de ataque: la aplicación sirve todo su tráfico solo por HTTP (puerto 80). Un atacante en la misma red (MITM, sniffing en redes compartidas o Wi-Fi abiertas) puede leer en texto plano las credenciales del login y el identificador de sesión (cookie PHPSESSID).

Impacto: pérdida de confidencialidad de credenciales y sesiones de estudiantes, docentes y personal administrativo; posible secuestro de sesión y acceso no autorizado a calificaciones y datos personales.

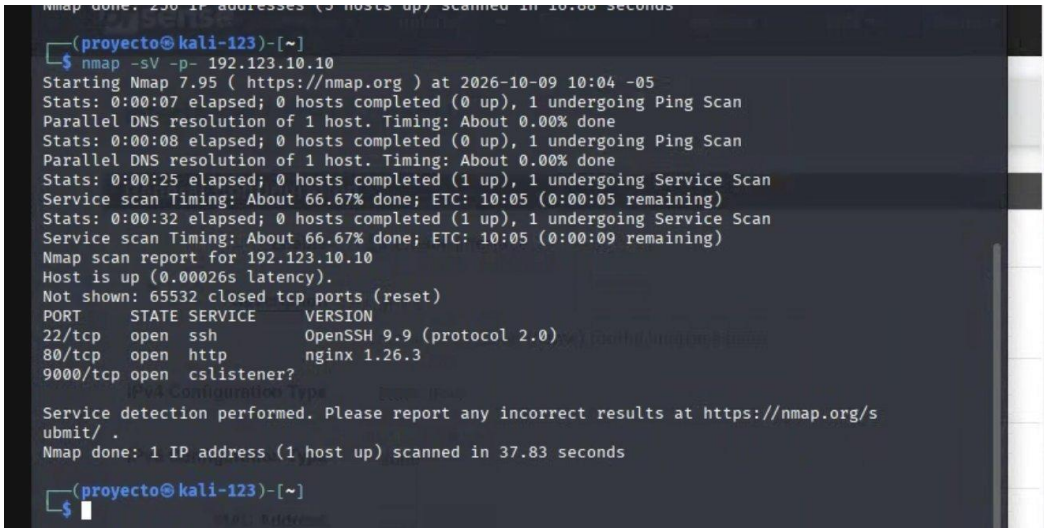


Figura 2.1. nmap -sV -p- 192.123.10.10: abiertos 22 (SSH), 80 (nginx) y 9000. No hay puerto 443: no existe transporte cifrado.

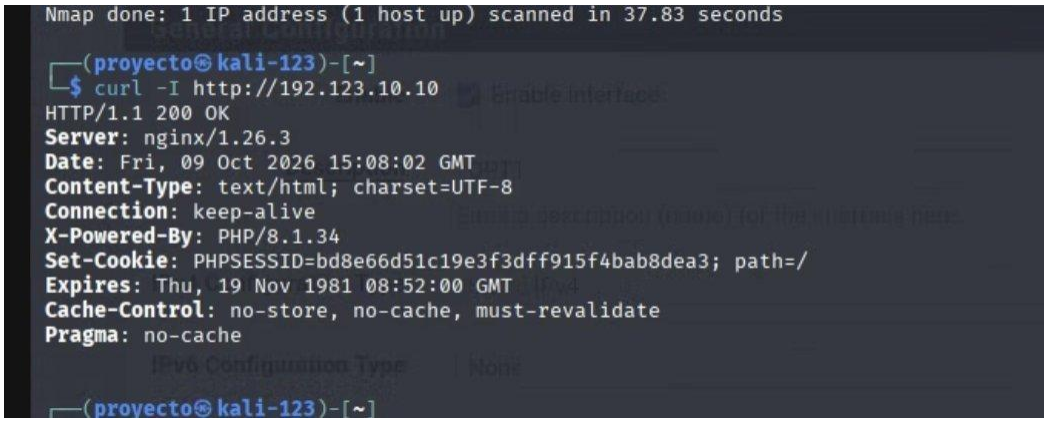


Figura 2.2. curl -I http://192.123.10.10: respuesta 200 sin redirección a HTTPS, sin cabeceras de seguridad y con Set-Cookie: PHPSESSID=...; path=/.

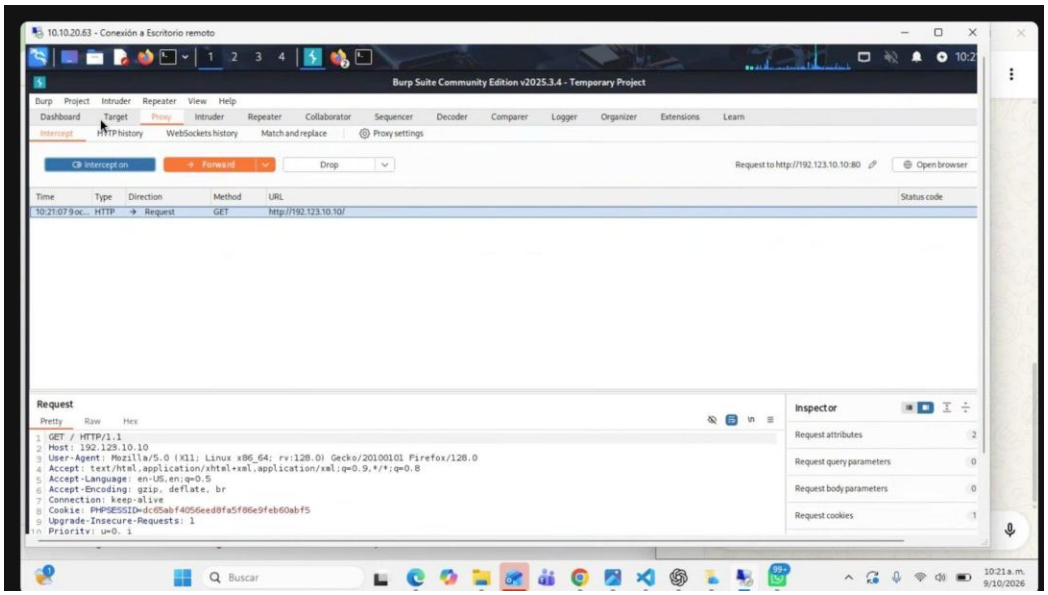


Figura 2.3. Burp Suite (Proxy > Intercept): la petición GET y la cookie PHPSESSID viajan en texto plano por el puerto 80.

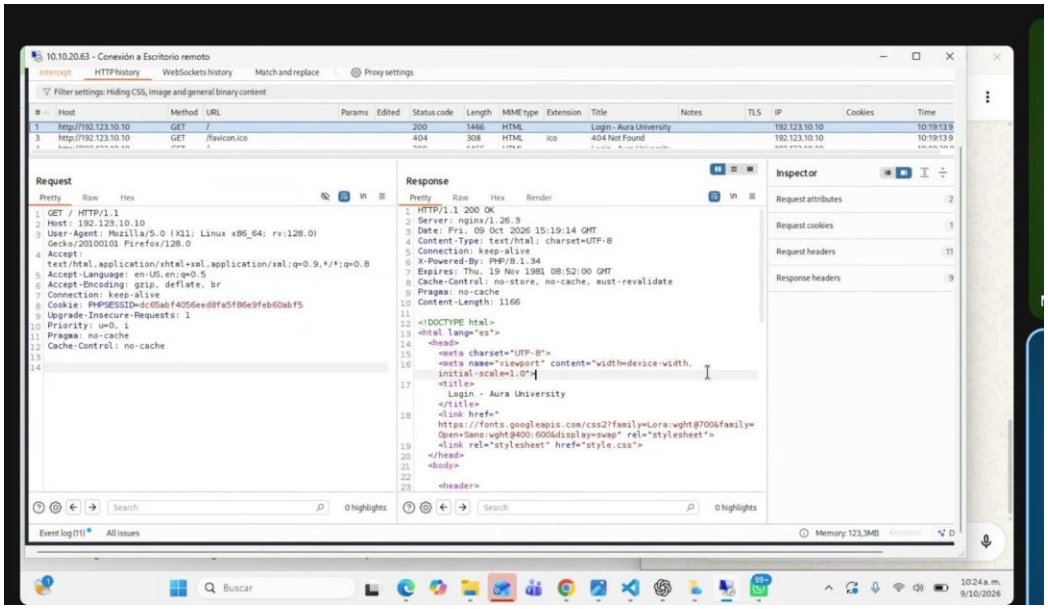


Figura 2.4. Burp HTTP history: petición y respuesta completas en claro (cabeceras Server y X-Powered-By incluidas).

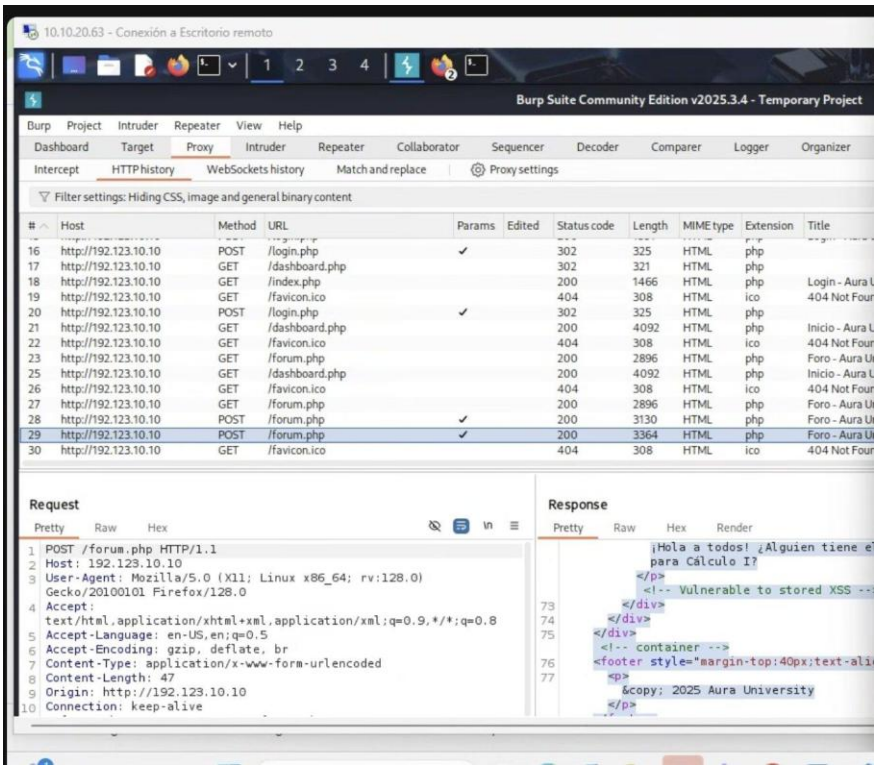


Figura 2.5. Burp HTTP history: las peticiones POST a /login.php y /forum.php también viajan por HTTP (http://192.123.10.10).

Evidencia complementaria: en la Fig. 2.7 se ve el cuerpo completo de un POST (content=%3Cscript%3E...) junto con la cookie PHPSESSID enviados a http://192.123.10.10 sin cifrar. El formulario de login usa el mismo canal (POST /login.php por HTTP, Fig. 2.5) y Firefox ofrece guardar la contraseña para http://192.123.10.10 (Fig. 1.5), por lo que usuario y contraseña viajan igualmente en texto plano y son legibles para cualquier interceptor, como lo hace Burp.

Vulnerabilidad 2: Cabeceras de seguridad ausentes y cookie de sesión mal configurada

Vector de ataque: la respuesta no incluye Strict-Transport-Security, Content-Security-Policy, X-Frame-Options ni X-Content-Type-Options, y la cookie PHPSESSID se emite sin Secure, HttpOnly ni SameSite (solo path=/). Esto habilita clickjacking, MIME-sniffing, lectura de la cookie desde JavaScript (agravado por V3) y envío de la cookie por canales no cifrados o en peticiones cross-site.

Impacto: robo de sesión mediante JavaScript malicioso, envío de la cookie a terceros, ataques de interfaz superpuesta y ejecución de contenido no autorizado.

Evidencia: Figuras 2.2 y 2.4 (respuesta sin ninguna cabecera de seguridad y cookie solo con path=/).

Vulnerabilidad 3: Cross-Site Scripting (XSS) almacenado en el foro estudiantil

Vector de ataque: el módulo del foro (forum.php, campo content del formulario "Nuevo Mensaje") guarda lo publicado y lo imprime después con echo $row['content']; sin codificación de salida. Se demostró publicando <script>alert(1)</script>, que se ejecuta automáticamente en el navegador de cualquier usuario que visite el foro. El campo username sí se escapaba, pero content no.

Impacto: al ser almacenado, el payload se ejecuta para todos los visitantes sin que hagan clic en nada. Combinado con V2 (cookie sin HttpOnly), permite robar el PHPSESSID de estudiantes, docentes o cuentas administrativas y suplantar su sesión, además de redirigir a phishing, alterar la página o registrar pulsaciones. Es el hallazgo de mayor severidad.

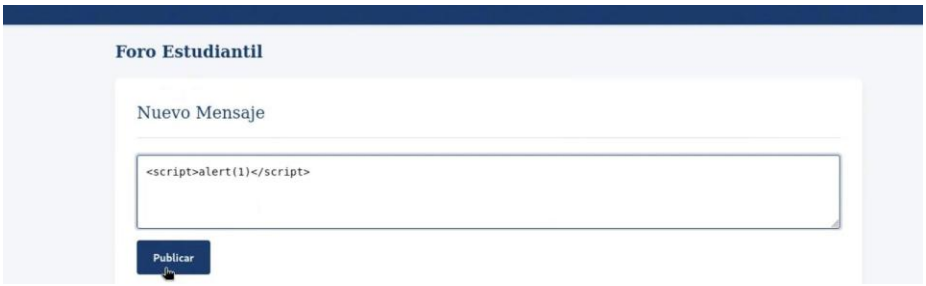


Figura 2.6. Payload ingresado en el formulario "Nuevo Mensaje" del foro.

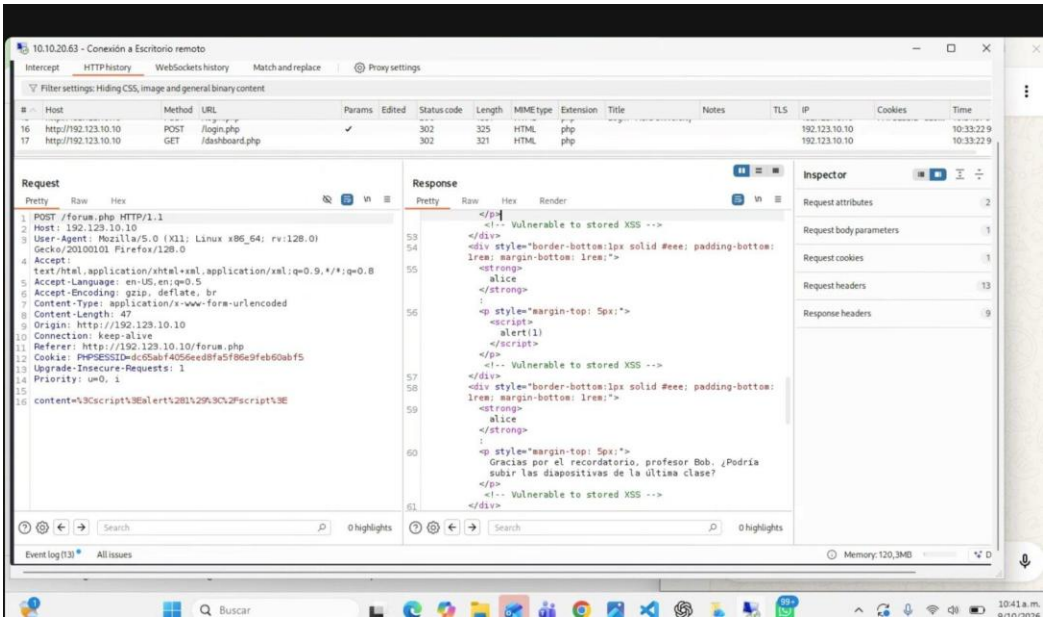


Figura 2.7. Burp HTTP history: POST /forum.php con content=%3Cscript%3Ealert(1)%3C%2Fscript%3E y respuesta que devuelve el script sin escapar (junto al comentario <!-- Vulnerable to stored XSS -->).

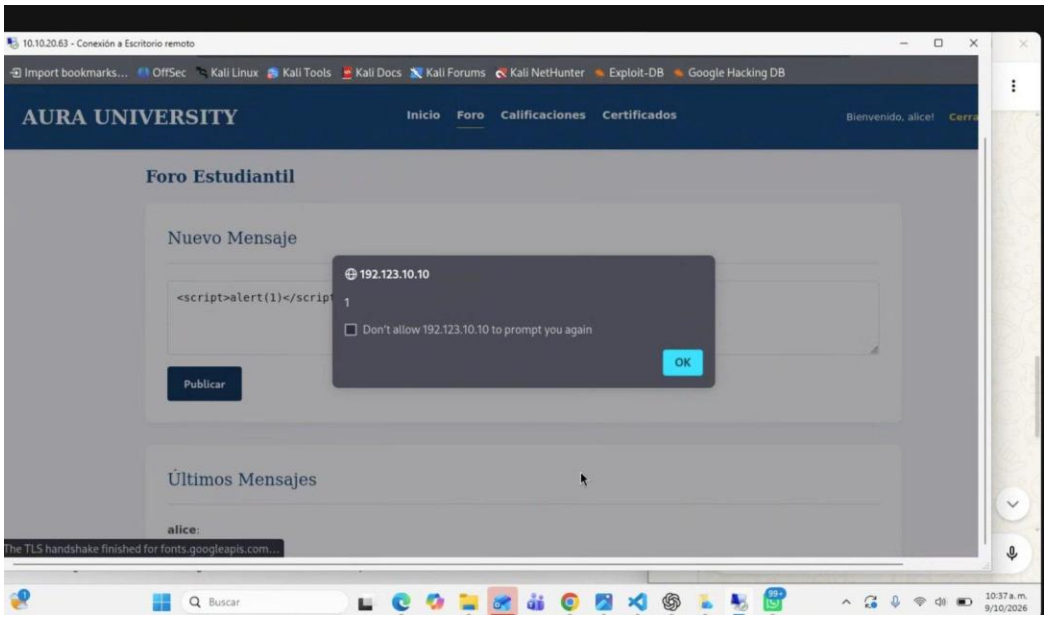


Figura 2.8. Explotación: el payload se ejecuta automáticamente al cargar el foro (ventana alert).

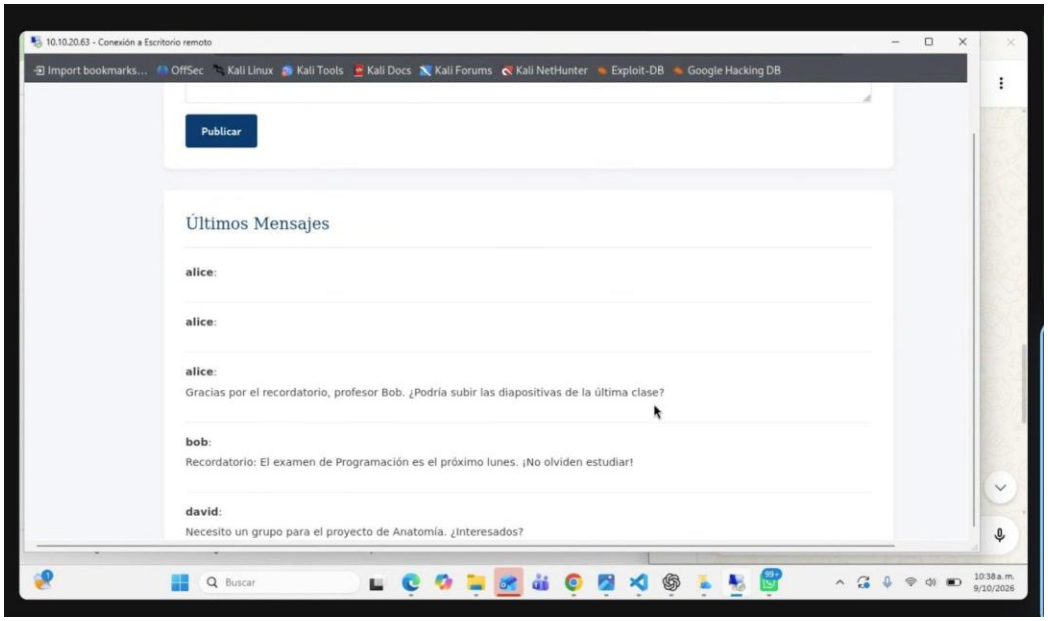


Figura 2.9. El mensaje queda almacenado en el foro y se vuelve a ejecutar en cada visita (XSS almacenado, no reflejado).

Vulnerabilidad 4: Puerto 9000/tcp (PHP-FPM / FastCGI) accesible desde la red

Hallazgo: en el escaneo completo (Fig. 2.1) el puerto 9000/tcp aparece abierto desde la Kali, con servicio sin identificar (cslistener?). La configuración original de nginx (Fig. 4.1) apunta a fastcgi_pass 127.0.0.1:9000, es decir, se esperaba que PHP-FPM escuchara solo en local. Que sea alcanzable desde otra red indica que el pool escucha en todas las interfaces.

Vector e impacto: FastCGI no autentica a quien le habla. Un atacante con alcance de red podría enviar peticiones FastCGI directamente, saltándose nginx y sus controles, y existen técnicas públicas para ejecutar código PHP por esta vía. **En este laboratorio la explotación no se probó**; se reporta como exposición confirmada con impacto potencial alto. No se aplicó mitigación en este ejercicio; la corrección propuesta (listen = 127.0.0.1:9000 más una regla de bloqueo en pfSense) está en la sección 8.

3. Priorización de riesgo

Se combinaron dos criterios: (1) severidad estimada con CVSS v3.1 (vector base, calculada con la fórmula de la especificación de FIRST) y (2) una matriz probabilidad × impacto con escala 1 a 5 (producto máximo 25), ajustada al contexto: portal académico con datos personales, acceso público y usuarios no técnicos.

#	Hallazgo	Vector CVSS 3.1 (estimado)	Score	Severidad	P	I	P×I / Prioridad
1	V3 — XSS almacenado (foro)	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N	8.7	ALTA	4	4	16 — 1.ª
2	V1 — Sin HTTPS	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	7.5	ALTA	3	4	12 — 2.ª
3	V4 — Puerto 9000 (PHP-FPM) expuesto	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H (potencial, no explotado)	8.1	ALTA (potencial)	2	5	10 — 3.ª
4	V2 — Cabeceras y cookie inseguras	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N	5.4	MEDIA	3	2	6 — 4.ª

Justificación: el XSS almacenado va primero porque es la única falla con explotación demostrada y automática contra terceros (Scope: Changed, pues compromete el navegador de la víctima) y porque su impacto se multiplica con la cookie sin HttpOnly. Su score es 8.7, es decir severidad **ALTA** (la categoría Crítica empieza en 9.0). La falta de HTTPS queda segunda por su amplio radio de afectación, aunque exige posición de red. El puerto 9000 tiene un CVSS potencial de 8.1 (AC:H porque la explotación no es trivial), pero se ubica tercero porque su explotación no fue demostrada; es de bajo esfuerzo corregirlo, por lo que se recomienda atenderlo pronto. Las cabeceras y la cookie quedan últimas: aisladas tienen menor explotabilidad, aunque son defensa en profundidad.

4. Endurecimiento

4.1 Resguardo de la configuración original

Antes de modificar se conservó una copia de /etc/nginx/nginx.conf (nginx.conf.bak, base de la comparación de la Fig. 4.3) y de /var/www/html/forum.php (forum.php.bak, Fig. 5.2), para permitir trazabilidad y reversión.

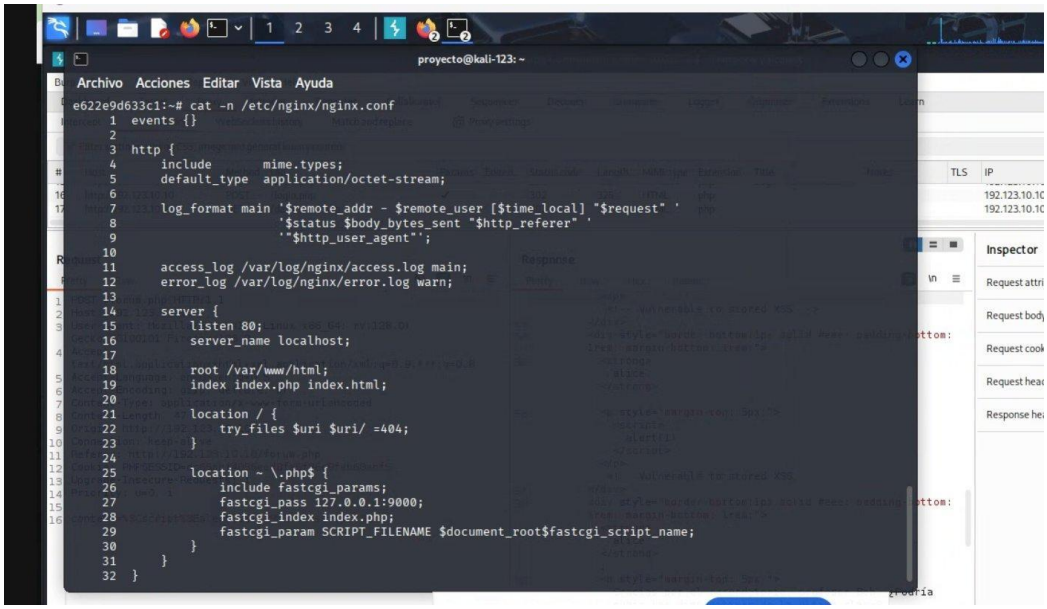


Figura 4.1. Configuración original de nginx: un solo bloque server en el puerto 80, sin TLS, con fastcgi_pass 127.0.0.1:9000.

Evidencia del respaldo: el diff -u contra /etc/nginx/nginx.conf.bak de la Fig. 4.3 solo es posible porque la copia existía antes de la edición; el respaldo de forum.php (cp a forum.php.bak) aparece en la Fig. 5.2 y se usa en la comparación de la Fig. 5.1. Con estas copias, revertir un cambio se reduce a restaurar el archivo .bak y ejecutar nginx -t && nginx -s reload.

4.2 Generación del certificado TLS

Se generó un certificado autofirmado (RSA 2048, 365 días) porque el entorno de laboratorio no dispone de una CA pública:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/server.key \
  -out /etc/nginx/ssl/server.crt \
  -subj "/C=CO/ST=Bogota/L=Bogota/O=AuraUniversity/CN=192.123.10.10"
```

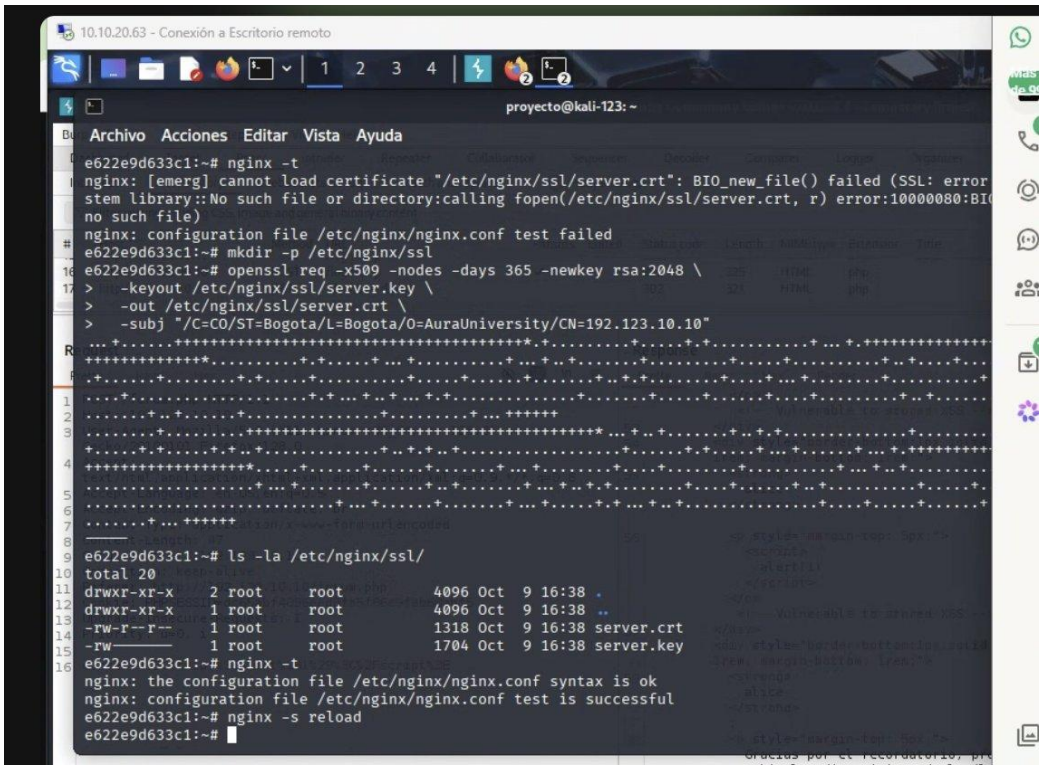


Figura 4.2. Generación del par de llaves y el certificado, validación de sintaxis (nginx -t: "syntax is ok") y recarga sin interrupción (nginx -s reload). El primer nginx -t falló por el certificado inexistente y pasó tras crearlo.

Por qué un certificado autofirmado no es recomendable en producción: ninguna CA de confianza lo respalda, así que el navegador no puede verificar la identidad del servidor ni construir una cadena de confianza; muestra advertencias (Figs. 4.4 y 4.5) y, sobre todo, **no protege contra un MITM activo**: un atacante puede presentar su propio certificado autofirmado y un usuario acostumbrado a "aceptar el riesgo" no lo notaría. En producción se debe usar un certificado de una CA pública (por ejemplo Let's Encrypt, gratuito y automatizable con ACME, RFC 8555), según NIST SP 800-52 Rev. 2.

Validación del certificado: la Fig. 4.5 confirma que el navegador recibe el certificado generado y lo rechaza con SEC_ERROR_UNKNOWN_ISSUER, es decir, el emisor es el propio servidor (autofirmado, CN=192.123.10.10). La negociación puede verificarse además desde la Kali con openssl s_client -connect 192.123.10.10:443 -brief, que muestra el protocolo (TLSv1.2/TLSv1.3) y un emisor igual al sujeto.

4.3 Configuración de nginx: redirección HTTPS y cabeceras

Se modificó nginx.conf para (1) redirigir todo el tráfico HTTP con código 301 a HTTPS, (2) habilitar un servidor en el puerto 443 con el certificado generado y solo TLS 1.2 y 1.3, y (3) añadir cinco cabeceras de seguridad. Fragmento relevante:

```
server {
```

```
    listen 80;
    server_name localhost;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name localhost;
    ssl_certificate     /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Content-Security-Policy "default-src 'self'; script-src 'self';
        style-src 'self' https://fonts.googleapis.com; font-src https://fonts.gstatic.com;" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    # root, index y location (/ y ~ \.php$) sin cambios
}
```

Ajuste	Qué mitiga	Referencia técnica
Redirección 301 HTTP→HTTPS	Tráfico en claro (V1); enlaces antiguos siguen funcionando	OWASP Transport Layer Security Cheat Sheet
ssl_protocols TLSv1.2 TLSv1.3	Elimina SSL/TLS obsoletos (downgrade, protocolos débiles)	RFC 8996; NIST SP 800-52 Rev. 2
Strict-Transport-Security	Fuerza HTTPS en visitas futuras y mitiga downgrade/stripping	RFC 6797
X-Frame-Options: DENY	Clickjacking (no permite incrustar la página en frames)	RFC 7034; OWASP Clickjacking Defense Cheat Sheet
X-Content-Type-Options: nosniff	MIME-sniffing: el navegador no reinterpreta el tipo de contenido	OWASP Secure Headers Project
Content-Security-Policy	Restringe el origen de scripts y estilos; reduce el impacto de un XSS (V3)	W3C CSP Level 3; OWASP CSP Cheat Sheet
Referrer-Policy	Limita la fuga de URLs en la cabecera Referer	W3C Referrer Policy; OWASP Secure Headers Project

Nota sobre HSTS: según RFC 6797 (8.1), los navegadores ignoran la cabecera Strict-Transport-Security cuando la conexión tiene errores de certificado. Con el certificado autofirmado de este

laboratorio la cabecera queda configurada, pero solo será efectiva en producción con un certificado válido.

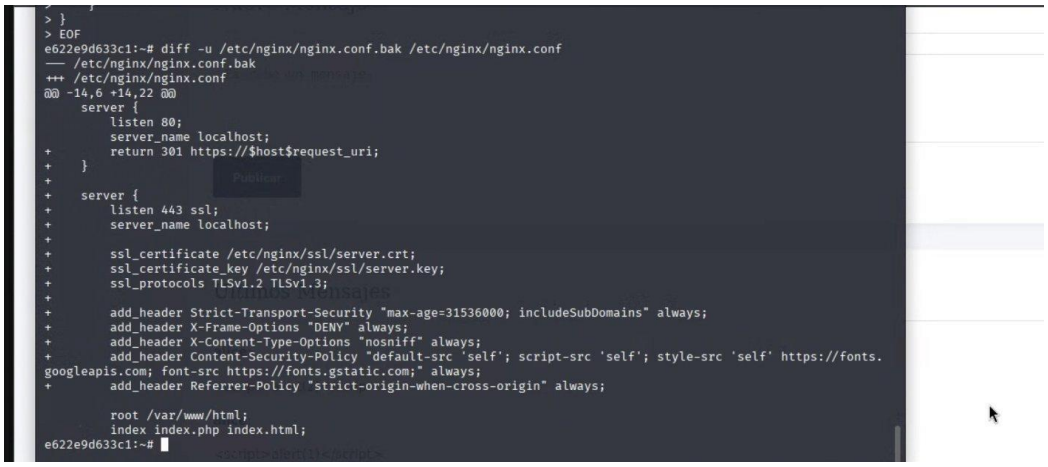


Figura 4.3. diff -u nginx.conf.bak nginx.conf: líneas añadidas (redirección 301, bloque 443, TLS 1.2/1.3 y cabeceras).

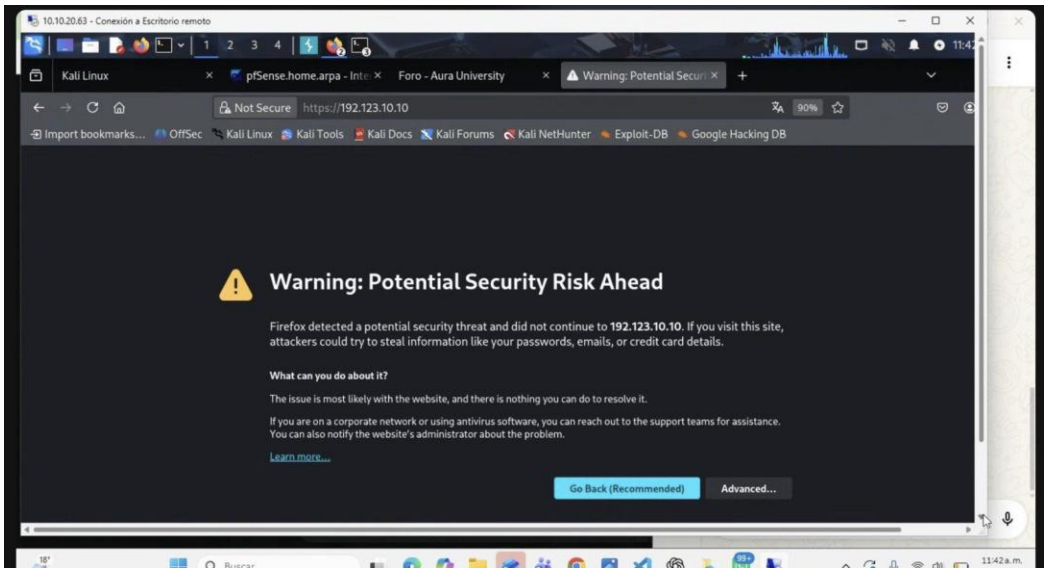


Figura 4.4. Advertencia de Firefox al abrir https://192.123.10.10 por el certificado autofirmado (respalda la justificación de la sección 4.2).

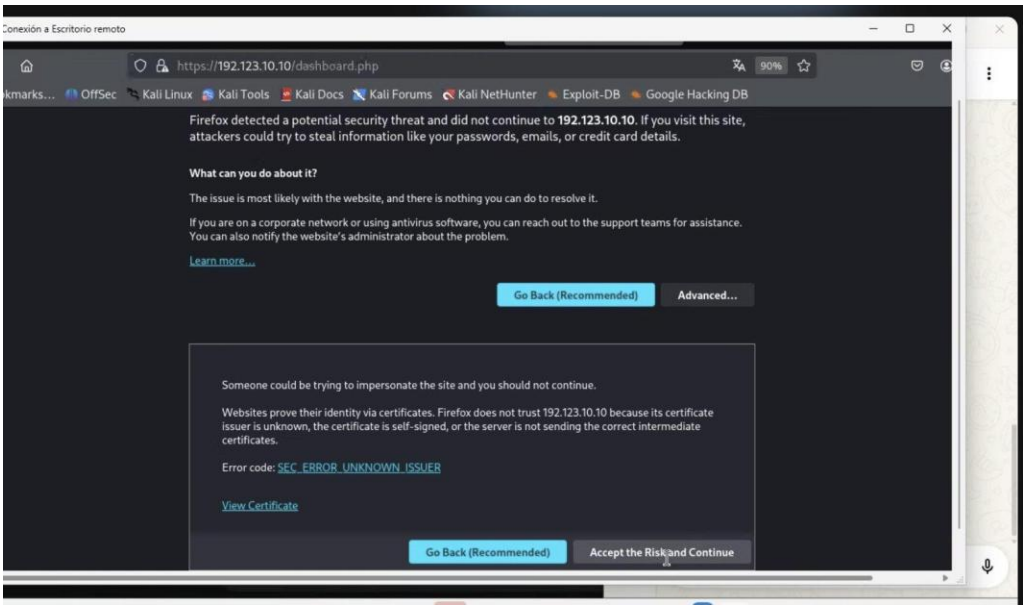


Figura 4.5. Detalle del error SEC_ERROR_UNKNOWN_ISSUER: emisor desconocido / certificado autofirmado.

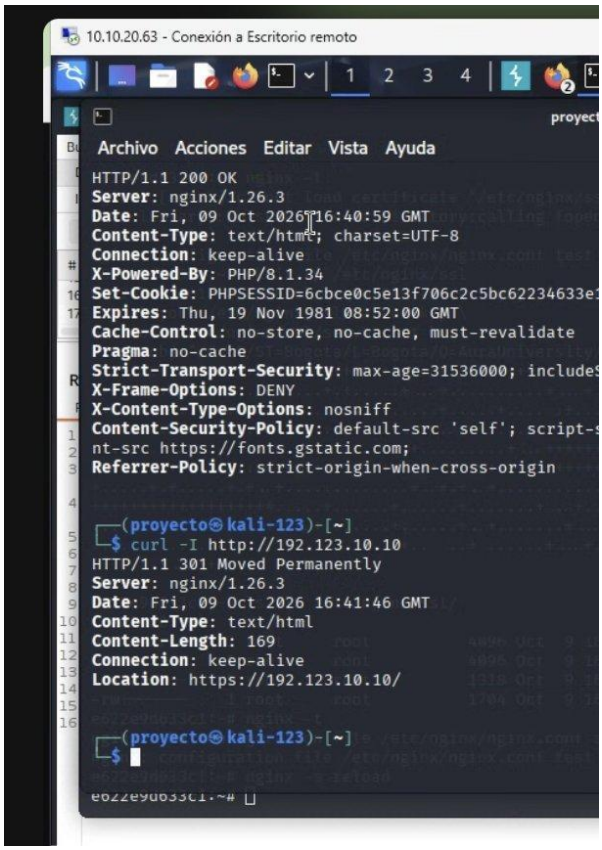


Figura 4.6. curl -I http://192.123.10.10 tras el cambio: 301 Moved Permanently con Location: https://192.123.10.10/.

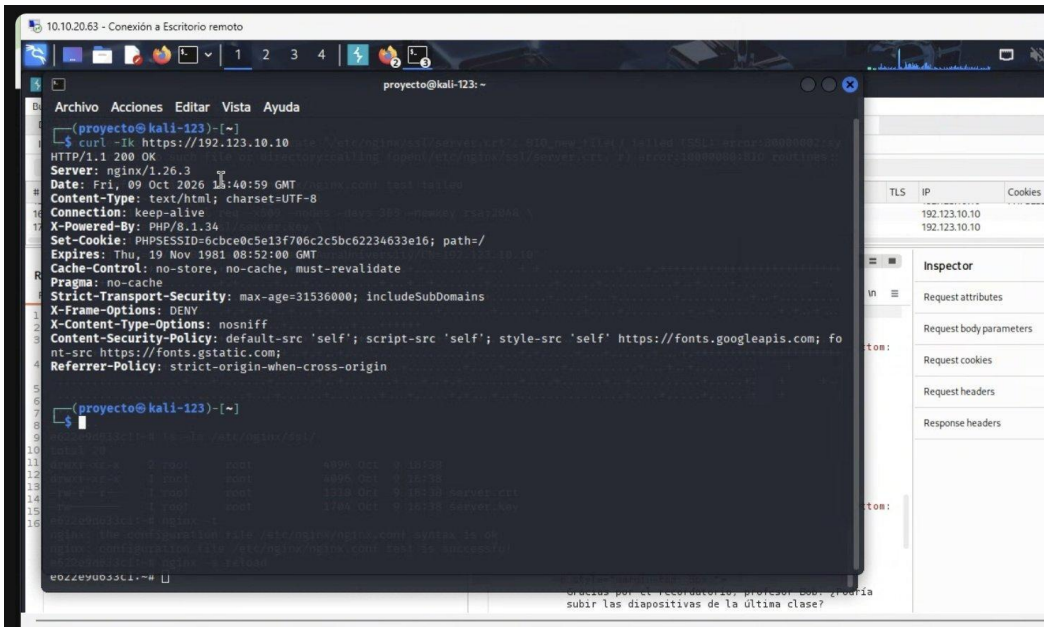


Figura 4.7. curl -Ik https://192.123.10.10: respuesta 200 con las cinco cabeceras de seguridad.

Disponibilidad del puerto 443: frente al escaneo inicial (Fig. 2.1), donde el 443 no estaba abierto, después del cambio el servidor responde por HTTPS con 200 OK (Fig. 4.7) y la aplicación se navega normalmente sobre https:// (Figs. 5.3 y 6.1), lo que confirma que el servicio quedó escuchando en 443/tcp y que HTTP solo redirige.

4.4 Endurecimiento de la cookie de sesión

Se añadió un archivo .user.ini en la raíz de la aplicación para forzar los atributos de la cookie de sesión de PHP sin modificar el código fuente:

```
session.cookie_secure = 1
session.cookie_httponly = 1
session.cookie_samesite = Strict
```

Tras recargar PHP-FPM (kill -USR2 sobre el proceso maestro) el cambio se confirmó en la cabecera Set-Cookie. Referencias: OWASP Session Management Cheat Sheet (atributos Secure, HttpOnly y SameSite) y CWE-614 / CWE-1004. Secure exige HTTPS, por eso este paso se hizo después de la migración.

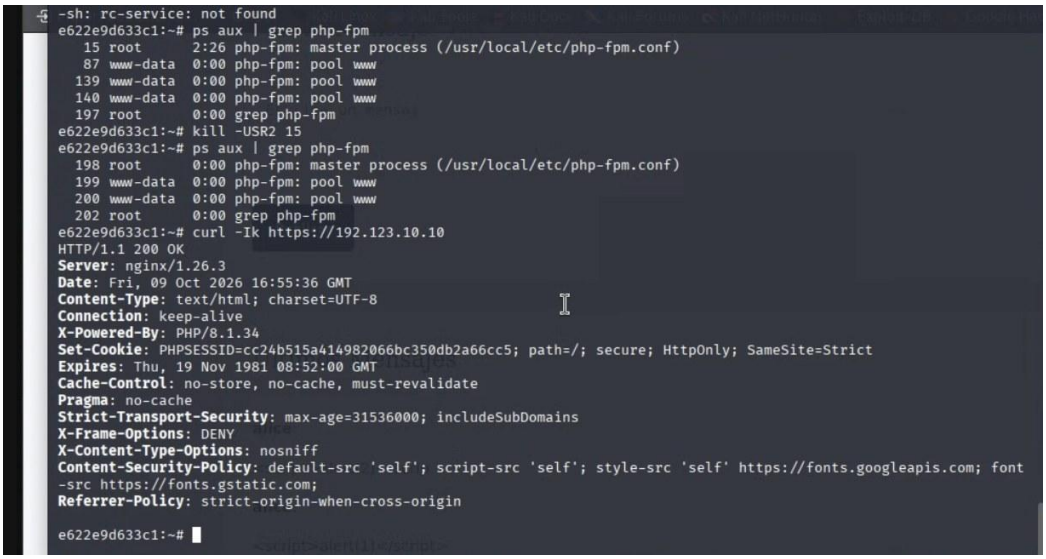


Figura 4.8. Recarga de PHP-FPM y curl -Ik https://...: la cookie PHPSESSID ahora incluye secure; HttpOnly; SameSite=Strict (compárese con la Fig. 2.2, donde solo tenía path=/).

5. Mitigación aplicada y validación posterior

Se corrigió la vulnerabilidad de mayor criticidad (V3, XSS almacenado) aplicando codificación de salida con htmlspecialchars() al contenido de los mensajes antes de renderizarlos. Es la misma técnica que el desarrollador ya usaba en username y que faltaba en content. Referencias: OWASP XSS Prevention Cheat Sheet, CWE-79 y OWASP A03:2021.

Antes:

```
<p style="margin-top: 5px;"><?php echo $row['content']; ?></p>
```

Después:

```
<p style="margin-top: 5px;"><?php echo htmlspecialchars($row['content']); ?></p>
```

En PHP 8.1 los flags por defecto de htmlspecialchars ya incluyen ENT_QUOTES | ENT_SUBSTITUTE; aun así se recomienda declararlos de forma explícita junto con la codificación 'UTF-8'.



Figura 5.1. diff -u forum.php.bak forum.php: una sola línea modificada (cambio mínimo, bajo riesgo de regresión).

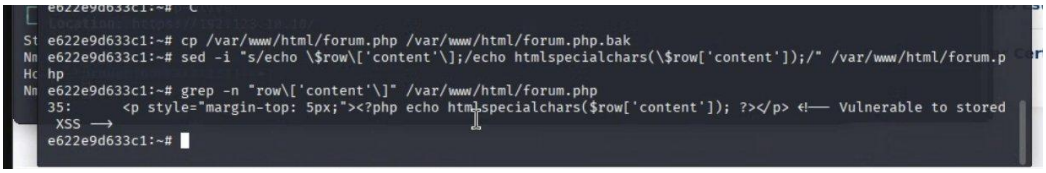


Figura 5.2. Respaldo de forum.php (cp), aplicación del cambio (sed -i) y verificación con grep.

5.1 Prueba comparativa antes / después del XSS

Momento	Payload enviado	Resultado observado
Antes de la mitigación	<script>alert(1)</script>	El script se ejecuta: aparece la ventana alert al cargar el foro (Fig. 2.8).
Después de la mitigación	<script>alert(2)</script>	El payload se muestra como texto literal; no se ejecuta ningún script (Fig. 5.3). Los mensajes antiguos también quedan neutralizados, porque el escape se hace al renderizar y no al guardar.

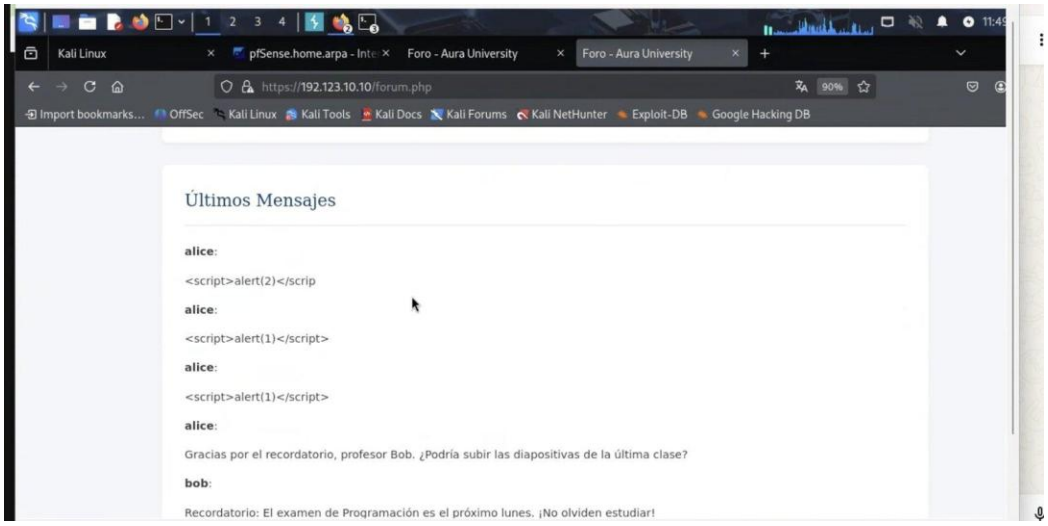


Figura 5.3. Validación posterior sobre HTTPS: el payload se ve como texto plano y no se ejecuta; los mensajes con alert(1) anteriores también aparecen escapados.

5.2 Resumen comparativo global del estado inicial y final

Aspecto	Antes	Después	Estado
V1 — Transporte	Puerto 443 cerrado (Fig. 2.1); HTTP 200 sin redirección (Fig. 2.2)	HTTP responde 301 a HTTPS (Fig. 4.6); HTTPS 200 (Fig. 4.7)	Mitigada (cert. autofirmado)
V2 — Cabeceras	Ninguna cabecera de seguridad (Fig. 2.2)	Cinco cabeceras presentes (Fig. 4.7)	Mitigada

Aspecto	Antes	Después	Estado
V2 — Cookie	PHPSESSID; path=/ (Fig. 2.2)	secure; HttpOnly; SameSite=Strict (Fig. 4.8)	Mitigada
V3 — XSS almacenado	alert(1) se ejecuta (Fig. 2.8)	Se muestra como texto (Fig. 5.3)	Mitigada
V4 — Puerto 9000	Abierto y accesible (Fig. 2.1)	Sin cambios	No mitigada (sección 8)
Versiones expuestas	Server: nginx/1.26.3 y X-Powered-By: PHP/8.1.34	Siguen visibles (Fig. 4.7)	No mitigada (sección 8)

6. Validación del servicio e impacto funcional

Tras aplicar migración a HTTPS, cabeceras, cookie segura y corrección del XSS, el servicio sigue operativo: el inicio de sesión funciona, y el panel de estudiante y el foro cargan sin errores sobre HTTPS.

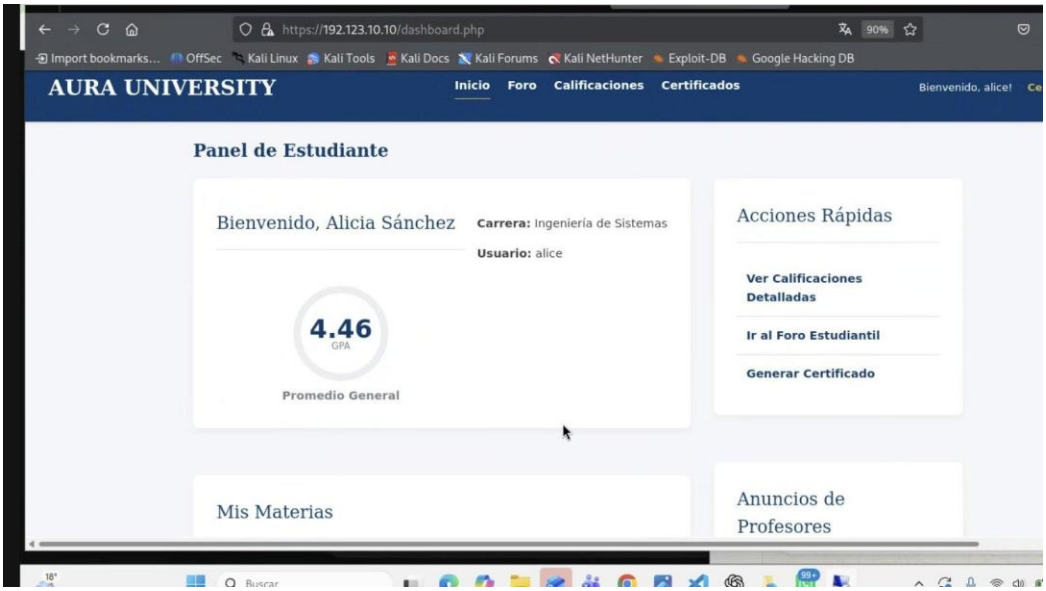


Figura 6.1. Panel de estudiante cargando con normalidad sobre HTTPS tras el endurecimiento (sesión de "alice").

6.1 Evaluación del impacto funcional

- **Rendimiento:** el sobrecosto de TLS con RSA 2048 es bajo para el volumen de un entorno académico y no se percibió degradación al navegar. No se midió de forma cuantitativa; se sugiere comparar tiempos con curl -o /dev/null -sk -w "%{time_total}\n" sobre HTTP y HTTPS.

- **Compatibilidad:** la redirección 301 mantiene operativos los enlaces y marcadores http:// existentes. Los clientes deben soportar TLS 1.2 o superior (se deshabilitaron versiones anteriores).
- **Experiencia de usuario:** el único impacto visible es la advertencia por certificado no confiable (Figs. 4.4 y 4.5), propia del certificado autofirmado; desaparece con una CA pública. SameSite=Strict puede pedir un nuevo inicio de sesión al llegar desde enlaces externos (por ejemplo, desde un correo).
- **Integraciones:** la CSP contempla Google Fonts (style-src y font-src) para no romper el estilo. Recursos de terceros no previstos serán bloqueados, y los atributos style="" en línea del foro podrían verse afectados por style-src 'self'; conviene revisar la consola del navegador (F12) y mover esos estilos a la hoja CSS.
- **HSTS con includeSubDomains:** al activarlo en producción afectará a todos los subdominios, que deberán servirse por HTTPS.

7. Riesgos residuales

Riesgos que persisten después de las correcciones, en orden de prioridad:

#	Riesgo residual	Descripción y evidencia	Prioridad	Estado
1	Certificado autofirmado	Sin cadena de confianza validada por una CA; un atacante en posición de red podría intentar un MITM si el usuario ignora la advertencia (Fig. 4.4). HSTS no es efectivo con este certificado.	Alta	Verificado
2	Puerto 9000 (PHP-FPM) expuesto	Sigue accesible desde la red (V4, Fig. 2.1). Impacto potencial alto; explotación no probada.	Alta	Verificado
3	PHP 8.1 sin soporte de seguridad	El stack usa PHP 8.1.34. Según el calendario oficial de PHP, la rama 8.1 dejó de recibir parches de seguridad el 31 de diciembre de 2025 (php.net/supported-versions.php), así que hoy no recibe correcciones de seguridad; también conviene confirmar que nginx 1.26.3 esté en la última versión de su rama. OWASP A06:2021.	Media-Alta	Verificado
4	Ausencia de protección CSRF	El formulario de login no incluye token anti-CSRF (Fig. 7.1) y el POST del foro envía solo el parámetro content (Fig. 2.7). No se probó la explotación.	Media-Alta	Por inspección

#	Riesgo residual	Descripción y evidencia	Prioridad	Estado
5	Sin límite de intentos de login	No se observó bloqueo, pero solo se hizo un intento fallido (Fig. 1.5); falta una prueba con varios intentos para confirmarlo.	Media	No verificado
6	Versiones expuestas	Server: nginx/1.26.3 y X-Powered-By: PHP/8.1.34 siguen en las respuestas (Fig. 4.7) y facilitan el fingerprinting.	Media-Baja	Verificado
7	Validación de entrada en el foro	El XSS quedó mitigado en la salida, pero no hay límite de longitud ni validación de entrada; posible spam o enlaces de phishing en texto.	Baja-Media	Por inspección
8	Comentarios de depuración	El comentario <!-- Vulnerable to stored XSS --> sigue en el código y en la respuesta (Figs. 2.7 y 5.1); revela la falla y la falta de limpieza antes de desplegar.	Baja	Verificado

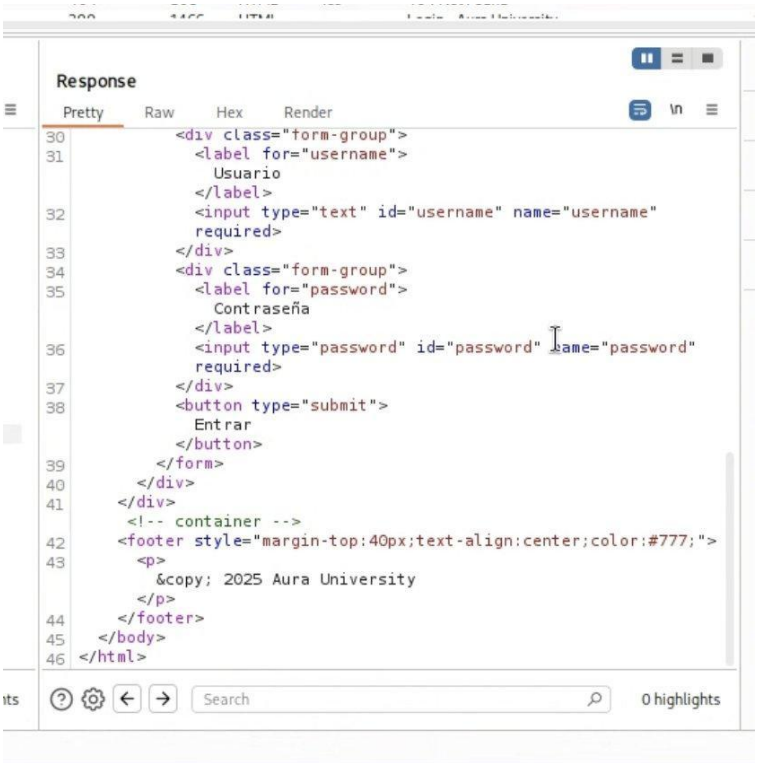


Figura 7.1. HTML del login en Burp: el formulario solo contiene username y password, sin campo de token anti-CSRF.

8. Recomendaciones

#	Recomendación	Prioridad	Esfuerzo	Referencia
1	Reemplazar el certificado autofirmado por uno de una CA pública (Let's Encrypt / certbot) con renovación automática.	Alta	Bajo	OWASP A02:2021; NIST SP 800-52r2; RFC 8555
2	Hacer que PHP-FPM escuche solo en local (listen = 127.0.0.1:9000 o socket Unix) y bloquear el 9000 en pfSense.	Alta	Bajo	OWASP A05:2021; CWE-668
3	Implementar tokens CSRF (token sincronizado) en login y en formularios que cambian estado, y mantener SameSite.	Alta	Medio	OWASP A01:2021; CWE-352; CSRF Prevention Cheat Sheet
4	Aplicar codificación de salida de forma central en toda la aplicación (motor de plantillas con escape automático) en vez de llamadas manuales a htmlspecialchars().	Alta	Medio	OWASP A03:2021; XSS Prevention Cheat Sheet
5	Actualizar PHP a una rama con soporte de seguridad y mantener nginx parchado; monitorear CVEs.	Alta	Medio	OWASP A06:2021; NIST SP 800-40r4
6	Ocultar versiones: server_tokens off; en nginx y expose_php = Off en php.ini.	Media	Bajo	OWASP A05:2021; OWASP WSTG (Information Gathering)
7	Limitar intentos de login (limit_req en nginx y bloqueo temporal en la aplicación).	Media	Medio	OWASP A07:2021
8	Endurecer SSH: PermitRootLogin no, autenticación por llaves y restricción de origen.	Media	Bajo	NIST SP 800-123
9	Integrar SAST y DAST (OWASP ZAP) en el pipeline CI/CD antes de cada despliegue.	Media	Alto	OWASP SAMM; DevSecOps
10	Eliminar comentarios de depuración y metadatos sensibles en un paso automatizado de build.	Baja	Bajo	Buenas prácticas de SDLC seguro

Configuraciones propuestas (**no aplicadas** en este ejercicio):

```
# nginx (bloque http o server)
server_tokens off;
```

```
# php.ini
expose_php = Off

# pool de PHP-FPM (por ejemplo www.conf)
listen = 127.0.0.1:9000
```

9. Respuestas a las preguntas

1. ¿Qué tipo de vulnerabilidades identificó y cómo podrían afectar a la organización si fueran explotadas?

Se identificaron cuatro: (a) transporte no cifrado, que expone en texto plano credenciales y cookies de sesión de la comunidad universitaria; (b) cabeceras de seguridad ausentes y cookie de sesión insegura, que amplían la superficie frente a clickjacking, fuga de información y robo de sesión; (c) un XSS almacenado en el foro, el más crítico, que ejecuta código en el navegador de cualquier visitante; y (d) el puerto 9000 de PHP-FPM accesible desde la red, con impacto potencial alto. En conjunto, un atacante podría interceptar credenciales, secuestrar sesiones de estudiantes, docentes o administrativos, acceder a calificaciones y datos personales protegidos por la normativa de protección de datos, y dañar la reputación de la universidad.

2. ¿Qué criterios utilizó para priorizar las vulnerabilidades encontradas?

Se combinaron la severidad estimada con CVSS v3.1 (verificada con la fórmula oficial) y una matriz probabilidad × impacto (escala 1 a 5), ajustada al contexto de un portal académico público con usuarios no técnicos. El XSS almacenado quedó primero (8.7, P×I 16) por tener explotación demostrada, afectar a terceros sin interacción adicional y amplificar el impacto de la cookie insegura; la falta de HTTPS segundo (7.5, P×I 12) por su amplio radio de afectación; el puerto 9000 tercero (8.1 potencial, P×I 10) porque la explotación no se demostró; y cabeceras y cookie al final (5.4, P×I 6) por ser defensa en profundidad más que un vector directo.

3. ¿Qué decisiones tomó al momento de fortalecer el servicio y qué riesgos residuales podrían mantenerse?

Se priorizaron cambios de alto impacto en seguridad y bajo riesgo para la disponibilidad: migración a HTTPS con redirección 301 (en lugar de cortar HTTP sin alternativa), cabeceras calibradas para no bloquear recursos legítimos como Google Fonts, endurecimiento de la cookie sin tocar el código (.user.ini) y una corrección mínima del XSS (una sola línea) para evitar regresiones. Antes de cada cambio se respaldaron los archivos y se validó la sintaxis con nginx -t. Se usó un certificado autofirmado por no haber CA pública disponible, y se documentó como riesgo. Persisten: el certificado autofirmado, el puerto 9000 expuesto, la ausencia de CSRF y de límite de intentos de login, las versiones visibles, PHP 8.1 sin soporte de seguridad y la necesidad de extender la sanitización a toda la aplicación.

4. ¿Cómo podría integrarse la seguridad dentro del ciclo de desarrollo de la aplicación para evitar este tipo de fallos en el futuro?

Con un ciclo de vida de desarrollo seguro (Secure SDLC / DevSecOps): (a) lineamientos de codificación segura y revisión de código entre pares enfocada en entrada y salida de datos; (b) análisis estático (SAST) en cada commit o pull request; (c) análisis dinámico (DAST, por ejemplo OWASP ZAP) en el pipeline antes de cada despliegue; (d) gestión de dependencias y versiones con monitoreo de CVEs; (e) pruebas de penetración periódicas antes de releases mayores; y (f) plantillas de infraestructura "seguras por defecto" (HTTPS, cabeceras y cookies seguras desde el inicio, y servicios internos solo en local).

5. ¿Qué recomendaciones haría al equipo de desarrollo y operaciones para mejorar la seguridad del servicio de manera continua?

Al equipo de desarrollo: usar un framework o motor de plantillas que escape la salida por defecto, implementar tokens CSRF, limitar intentos de login y revisar el código contra el OWASP Top 10 antes de cada fusión. Al equipo de operaciones: automatizar la renovación de certificados con una CA pública, tener un proceso formal de gestión de parches (SO, nginx, PHP), restringir los servicios internos (como PHP-FPM) a local, endurecer SSH, monitorear intentos de acceso anómalos y mantener copias versionadas de las configuraciones críticas, como se hizo en este ejercicio. Ambos equipos deberían institucionalizar escaneos de seguridad periódicos y no solo reactivos.

10. Conclusiones

El ejercicio permitió identificar y documentar cuatro hallazgos sobre la aplicación "Aura University" en el segmento DMZ: transporte no cifrado, cabeceras y cookie inseguras, un XSS almacenado con explotación demostrada y el puerto 9000 de PHP-FPM expuesto. Se aplicó el endurecimiento (migración a HTTPS con redirección 301, cinco cabeceras de seguridad, TLS 1.2/1.3 y cookie de sesión segura) y se corrigió el XSS con validación comparativa antes/después, sin afectar la disponibilidad ni la funcionalidad. Quedan documentados los riesgos residuales y un plan de recomendaciones priorizadas por criticidad y esfuerzo para seguir reduciendo la superficie de ataque.

Referencias

[1] OWASP Top 10:2021 — https://owasp.org/Top10/

[2] OWASP Secure Headers Project — https://owasp.org/www-project-secure-headers/

[3] OWASP Cheat Sheet Series: Session Management, Cross Site Scripting Prevention, CSRF Prevention, Transport Layer Security, Content Security Policy, Clickjacking Defense — https://cheatsheetseries.owasp.org/

[4] FIRST. Common Vulnerability Scoring System v3.1: Specification Document — https://www.first.org/cvss/v3.1/specification-document

[5] MITRE CWE: CWE-79, CWE-319, CWE-352, CWE-614, CWE-668, CWE-693, CWE-1004 — https://cwe.mitre.org/

[6] RFC 6797 — HTTP Strict Transport Security (HSTS) — https://www.rfc-editor.org/rfc/rfc6797

[7] RFC 7034 — HTTP Header Field X-Frame-Options — https://www.rfc-editor.org/rfc/rfc7034

[8] RFC 8996 — Deprecating TLS 1.0 and TLS 1.1 — https://www.rfc-editor.org/rfc/rfc8996

[9] RFC 8555 — Automatic Certificate Management Environment (ACME) — https://www.rfc-editor.org/rfc/rfc8555

[10] NIST SP 800-52 Rev. 2 — Guidelines for the Selection, Configuration, and Use of TLS Implementations — https://csrc.nist.gov/pubs/sp/800/52/r2/final

[11] NIST SP 800-40 Rev. 4 — Guide to Enterprise Patch Management Planning — https://csrc.nist.gov/pubs/sp/800/40/r4/final

[12] NIST SP 800-123 — Guide to General Server Security — https://csrc.nist.gov/pubs/sp/800/123/final

[13] W3C Content Security Policy Level 3; W3C Referrer Policy — https://www.w3.org/TR/CSP3/ · https://www.w3.org/TR/referrer-policy/

[14] PHP Manual: htmlspecialchars, configuración de sesiones y versiones soportadas — https://www.php.net/manual/ · https://www.php.net/supported-versions.php

COMANDOS

```
# Fase A: análisis (desde la Kali)
ip a
nmap -sn 192.123.99.0/24
nmap -sn 192.123.10.0/24
nmap -sV -p- 192.123.10.10
curl -I http://192.123.10.10

# Fase B: endurecimiento (en DMZ.10 por SSH)
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
cp /var/www/html/forum.php /var/www/html/forum.php.bak
mkdir -p /etc/nginx/ssl
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/server.key -out /etc/nginx/ssl/server.crt \
  -subj "/C=CO/ST=Bogota/L=Bogota/O=AuraUniversity/CN=192.123.10.10"
# edición de /etc/nginx/nginx.conf (sección 4.3) y creación de .user.ini (sección 4.4)
nginx -t && nginx -s reload
ps aux | grep php-fpm ; kill -USR2 <PID del proceso maestro>
sed -i "s/echo \$row\['content'\];/echo htmlspecialchars(\$row['content']);/"
/var/www/html/forum.php
grep -n "row\['content'\]" /var/www/html/forum.php

# Verificación posterior
curl -I http://192.123.10.10      # 301 a HTTPS
curl -Ik https://192.123.10.10    # cabeceras y cookie
diff -u /etc/nginx/nginx.conf.bak /etc/nginx/nginx.conf
diff -u /var/www/html/forum.php.bak /var/www/html/forum.php
```